Вернуться к 'Рекурсия. Функции высших порядков'

| | |
|---|---|
| **Тест начат** | Пятница, 25 Сентябрь 2026, 22:46 |
| **Состояние** | Завершенные |
| **Завершен** | Пятница, 25 Сентябрь 2026, 23:22 |
| **Прошло времени** | 36 мин. 24 сек. |
| **Баллы** | 13,80/15,00 |
| **Оценка** | **4,60** из 5,00 (**92**%) |

Вопрос **1**

Верно

Баллов: 1,00 из 1,00

Используя следующее определение, покажите как вычисляется

`init [1, 2, 3]`

```
init :: [a] -> [a]
init [_] = []
init (x : xs) = x : init xs
```

Выберите один ответ:

a.

```
init [1,2,3]
= { applying init }
[1,2] : init [3]
= { applying init }
[1,2] : []
= { list notation }
[1,2]
```

b.

```
init [1,2,3]
= { applying init }
1 : init [2,3]
= { applying init }
1 : 2 : init [3]
= { applying init }
1 : 2 : 3 : init []
= { applying init }
1 : 2 : 3 : []
= { list notation }
[1,2,3]
```

c.

```
init [1,2,3]
= { applying init }
1 : init [2,3]
= { applying init }
1 : 2 : init [3]
= { applying init }
1 : 2 : []
= { list notation }
[1,2]
```

✔

d.

```
init [1,2,3]
= { applying init }
1 : init [2,3]
= { applying init }
1 : []
= { list notation }
[1]
```

Вопрос **2**

Верно

Баллов: 1,00 из 1,00

Выберите правильное определение для функции, которая объединяет список списков:

`concat :: [[a]] -> [a]`

Выберите один ответ:

```
concat [[]] = []
concat (xs : xss) = xs ++ concat xss
```

```
concat [] = []
concat (xs : xss) = xs : concat xss
```

```
concat [] = []
concat (xs : xss) = xs ++ concat xss
```

✔

```
concat [] = [[]]
concat (xs : xss) = xs ++ concat xss
```

Вопрос **3**

Верно

Баллов: 1,00 из 1,00

Выберите эквивалентое определение списка `[f x | x <- xs, p x]` через функции высших порядков

Выберите один ответ:

- `map p (map f xs)`
- `map f (filter p xs)` ✔
- `filter p (map f xs)`
- `map f (takeWhile p xs)`

Вопрос **4**

Верно

Баллов: 1,00 из 1,00

Выражение `filter even (map (+1) [1..5])` равно

Выберите один ответ:

- `[3,5]`
- `[2,4,6]` ✔
- `[]`
- `[2,4]`
- `[1,3,5]`

Вопрос **5**

Частично правильный

Баллов: 0,70 из 1,00

Выберите все варианты, которые реализуют функцию any

```
any :: (a -> Bool) -> [a] -> Bool
```

где списки являются конечными, правильно определенными списками и где предикат p всегда возвращает либо True либо False

Выберите один или несколько ответов:

- ☑ `any p xs = not (all (\ x -> not (p x)) xs)` ✔
- ☑ `any p xs = length (filter p xs) > 0` ✔
- ☐ `any p = map p . or`
- ☑ `any p = or . map p` ✔
- ☑ `any p xs = foldr (||) True (map p xs)` ✖
- ☐ `any p = null . filter p`
- ☑ `any p = not . null . dropWhile (not . p)` ✔

Вопрос **6**

Верно

Баллов: 1,00 из 1,00

Выберите все правильные определения оператора возведения в степень ^ для неотрицательных целых чисел (в том числе 0).

Выберите один или несколько ответов:

- [ ]
```
m ^ 1 = m
m ^ n = m * m ^ (n - 1)
```

- [ ]
```
m ^ 0 = 1
m ^ n = n * n ^ (m - 1)
```

- [x]
```
m ^ 0 = 1
m ^ n = m * m ^ (n - 1)
```

- [x]
```
m ^ 0 = 1
m ^ n = m * (^) m (n - 1)
```

- [ ]
```
m ^ 0 = 1
m ^ n = m * m ^ n - 1
```

- [ ]
```
m ^ 0 = 0
m ^ n = m * m ^ (n - 1)
```

- [ ]
```
m ^ 0 = 1
m ^ n = (m * m) ^ (n - 1)
```

- [x]
```
m ^ 0 = 1
m ^ n = m * m * m ^ (n - 2)
```

Вопрос **7**

Верно

Баллов: 1,00 из 1,00

Выберите правильное определение для функции, которая выдает список с n одинаковыми элементами:

```
replicate :: Int -> a -> [a]
```

Выберите один ответ:

- (•)
```
replicate 0 _ = []
replicate n x = x : replicate (n - 1) x
```

- ( )
```
replicate 0 _ = []
replicate n x = replicate (n - 1) x : x
```

- ( )
```
replicate 1 x = x
replicate n x = x : replicate (n - 1) x
```

- ( )
```
replicate 1 _ = []
replicate n x = replicate (n - 1) x ++ [x]
```

Вопрос **8**

Верно

Баллов: 1,00 из 1,00

Выберите корректное определение для функции, которая определяет, являются ли все элементы исходного списка True

```
and :: [Bool] -> Bool
```

Выберите один или несколько ответов:

- [x] ✔
```
and [] = True
and (b : bs) = b && and bs
```

- [ ]
```
and [] = False
and (b : bs) = b && and bs
```

- [x] ✔
```
and [] = True
and (b : bs) = and bs && b
```

- [x] ✖
```
and [] = True
and (b : bs)
  | b = b
  | otherwise = and bs
```

- [x] ✔
```
and [] = True
and (b : bs)
  | b = and bs
  | otherwise = False
```

- [ ]
```
and [] = False
and (b : bs) = b || and bs
```

- [x] ✔
```
and [] = True
and (b : bs)
  | b == False = False
  | otherwise = and bs
```

Вопрос **9**

Верно

Баллов: 1,00 из 1,00

Используя слдующее опрделеление, показать, как вычисляется

`drop 3 [1, 2, 3, 4, 5]`

```
drop :: Int -> [a] -> [a]
drop 0 xs = xs
drop n [] = []
drop n (_ : xs) = drop (n - 1) xs
```

Выберите один ответ:

a.

```
drop 3 [1,2,3,4,5]
= { applying drop }
drop 2 [2,3,4,5]
= { applying drop }
drop 1 [3,4,5]
= { applying drop }
drop 0 [4,5]
= { applying drop }
[5]
```

b.

```
drop 3 [1,2,3,4,5]
= { applying drop }
drop 2 [1,2,4,5]
= { applying drop }
drop 1 [1,4,5]
= { applying drop }
drop 0 [4,5]
= { applying drop }
[4,5]
```

c.

```
drop 3 [1,2,3,4,5]
= { applying drop }
drop 2 [2,3,4,5]
= { applying drop }
drop 1 [3,4,5]
= { applying drop }
drop 0 [4,5]
= { applying drop }
[4,5]
```

✔

d.

```
drop 3 [1,2,3,4,5]
= { applying drop }
drop 2 [1,2,3,4]
= { applying drop }
drop 1 [1,2,3]
= { applying drop }
drop 0 [1,2]
= { applying drop }
[1,2]
```

Вопрос **10**

Верно

Баллов: 1,00 из 1,00

Выберите все варианты, которые реализуют функцию dropWhile

```
dropWhile :: (a -> Bool) -> [a] -> [a]
```

где списки являются конечными, правильно определенными списками  и где предикат p всегда возвращает либо True либо False

Выберите один ответ:

```
dropWhile p = foldr (\ x acc -> if p x then acc else x : acc)
```

```
dropWhile p = foldl add []
  where add [] x = if p x then [] else [x]
        add acc x = x : acc
```

```
dropWhile _ [] = []
dropWhile p (x : xs)
  | p x = dropWhile p xs
  | otherwise = x : xs
```

✔

```
dropWhile _ [] = []
dropWhile p (x : xs)
  | p x = dropWhile p xs
  | otherwise = xs
```

Вопрос **11**

Верно

Баллов: 1,00 из 1,00

Выберите все варианты, которые реализуют функцию map

```
map :: (a -> b) -> [a] -> [b]
```

где списки являются конечными, правильно определенными списками  и где предикат p всегда возвращает либо True либо False

Выберите один ответ:

```
map f = foldr (\ x xs -> xs ++ [f x]) []
```

```
map f = foldl (\ xs x -> f x : xs) []
```

```
map f = foldr (\ x xs -> f x ++ xs) []
```

```
map f = foldl (\ xs x -> xs ++ [f x]) []
```

✔

Вопрос **12**

Частично правильный

Баллов: 0,10 из 1,00

Выберите все варианты, которые реализуют функцию all

```
all :: (a -> Bool) -> [a] -> Bool
```

где списки являются конечными, правильно определенными списками  и где предикат p всегда возвращает либо True либо False

Выберите один или несколько ответов:

- [x] `all p xs = foldr (&&) False (map p xs)` ✗
- [x] `all p = foldr (&&) True . map p` ✓
- [x] `all p xs = and (map p xs)` ✓
- [x] `all p xs = foldl (&&) True (map p xs)` ✓
- [x] `all p xs = map p (and xs)` ✗
- [x] `all p = map p . and` ✗
- [x] `all p = and . map p` ✓
- [x] `all p = not . any (not . p)` ✓

Вопрос **13**

Верно

Баллов: 1,00 из 1,00

Какие из предложенных выражений эквивалентны [1, 2, 3, 4]:

Выберите один ответ:

- `map (<= 4) [1..]`
- `1:2:3:4`
- `take 4 (iterate (+1) 1)` ✓
- `filter (<= 4) (repeat 1)`
- `1++2++3++4`

Вопрос **14**

Верно

Баллов: 1,00 из 1,00

Выберите все варианты, которые реализуют функцию filter

```
filter :: (a -> Bool) -> [a] -> [a]
```

где списки являются конечными, правильно определенными списками  и где предикат p всегда возвращает либо True либо False

Выберите один ответ:

- 
```
filter p = foldl (\ xs x -> if p x then x : xs else xs) []
```
- 
```
filter p = foldr (\ x xs -> if p x then xs ++ [x] else xs) []
```
- 
```
filter p = foldl (\ x xs -> if p x then xs ++ [x] else xs) []
```
- 
```
filter p = foldr (\ x xs -> if p x then x : xs else xs) []
```
✓

Вопрос **15**

Верно

Баллов: 1,00 из 1,00

Выберите правильное определение для функции слияния

`merge :: Ord a => [a] -> [a] -> [a]`

которая объединяет два списка, отсортированных в порядке возрастания в результирующий отсортированный в порядке возрастания список. Например:

```
> merge [2, 5, 6] [1, 3, 4]
[1, 2, 3, 4, 5, 6]
```

Выберите один ответ:

```
merge [] ys = ys
merge xs [] = xs
merge (x : xs) (y : ys)
  = if x <= y then y : merge xs (y : ys) else x : merge (x : x
```

```
merge [] ys = ys
merge xs [] = xs
merge (x : xs) (y : ys)
  = if x <= y then y : merge (x : xs) ys else x : merge xs (y
```

```
merge [] ys = ys
merge xs [] = xs
merge (x : xs) (y : ys)
  = if x <= y then x : merge xs (y : ys) else y : merge (x : x
```

✔

```
merge [] ys = ys
merge xs [] = xs
merge (x : xs) (y : ys)
  = if x <= y then x : merge xs ys else y : merge xs ys
```